



学习任务1: Picker 部分

熟悉 *Picker* 的基本用法

验证对象的代码和文档位于: [同步 FIFO 的设计规范和代码](#)。

准备工作

1. 创建项目目录结构:

```
fifo_test/  
├── rtl/      # 放置FIFO的RTL代码  
└── test_smoke.py # 测试用例文件
```

- 把同步 FIFO 的代码放在该 rtl 下, 命名为 SyncFIFO.v 。

1. DUT 类的创建和初始化

1. 创建同步 FIFO 的 DUT 类
2. 创建 test_reset_dut 函数作为的测试用例: 把 rst_n 信号拉低 5 个周期后, 再拉高 2 个周期, 导出波形信号
3. 查看波形, 关注: 在默认的写模式下, rst_n 的拉高是在是在什么时候生效的?
4. 把 rst_n 的写模式修改为立即模式 Imme, 再重新观察 rst_n 的拉高是在是在什么时候生效的? 跟默认情况下的波形做一个比对。

5. 思考：在对模块进行初始化时，仅仅是对 `rst_n` 进行赋值就足够了吗？还需要注意什么？

。可以阅读以下材料帮助你思考：

。 (4.1 Value set) Verilog HDL 的数值集合包含四种基本值：

- 0：表示逻辑零或假条件
- 1：表示逻辑一或真条件
- x：表示**未知逻辑值**
- z：表示**高阻态**

(4.2.1 Net declarations) `wire` 类型的初始值为 `z`

(4.2.2 Variable declarations) `reg` 类型的初始值为 `x`

—— 来自[IEEE Std 1364-2005, IEEE Standard for Verilog® Hardware Description Language](#)

。 Verilator 本质上是一个二态仿真器，所以对于 `x` 和 `z` 值会分配一个具体的常量值，可以是随机数也可以只固定值，具体取决于[相关选项](#)的控制。

—— 来自 Verilator 文档的 [Unknown States](#) 部分

2. 编写复位测试

在测试用例 `test_reset_dut` 的基础上，测试完成复位后，用 `assert` 判断 FIFO 的输出、读写指针是否为 0

3. 编写冒烟测试

冒烟测试是验证流程中的第一道防线，用于快速确认系统的基本功能是否正常工作。

这一术语源自硬件行业：当新硬件首次通电时，如果没有冒烟，则初步通过测试。

在芯片验证中，冒烟测试能够在开发早期发现严重问题，节省大量调试时间。

添加测试用例 `test_smoke_dut`，测试步骤如下所示：

1. 对 DUT 进行复位
2. 向 FIFO 分别写入两个数据：
 1. 给 `we_i` 置高，`data_i` 赋值为 `0x114`，保持一个周期，之后判断 `empty_o` 和 `full_o` 是否为 0
 2. 给 `we_i` 置高，`data_i` 赋值为 `0x514`，保持一个周期
3. 从 FIFO 读出两个数据：
 1. 给 `we_i` 置低、`re_i` 置高，保持一个周期，之后读取 `data_o`，判断结果是否为 `0x114`
 2. 给 `we_i` 置低、`re_i` 置高，保持一个周期，之后读取 `data_o`，判断结果是否为 `0x514`、`empty_o` 是否为 1

然后运行测试用例，验证 DUT 能否通过测试。